QUANTIFYING LANGUAGE MODELS' SENSITIVITY TO SPURIOUS FEATURES IN PROMPT DESIGN *or: How I learned to start worrying about prompt formatting*

Melanie Sclar[1] **Yejin Choi**[1,2] **Yulia Tsvetkov**[1] **Alane Suhr**[3]
[1]Paul G. Allen School of Computer Science & Engineering, University of Washington
[2]Allen Institute for Artificial Intelligence [3]University of California, Berkeley
msclar@cs.washington.edu

arXiv:2310.11324v2 [cs.CL] 1 Jul 2024

ABSTRACT

As large language models (LLMs) are adopted as a fundamental component of language technologies, it is crucial to accurately characterize their performance. Because choices in prompt design can strongly influence model behavior, this design process is critical in effectively using any modern pre-trained generative language model. In this work, we focus on LLM sensitivity to a quintessential class of meaning-preserving design choices: prompt formatting. We find that several widely used open-source LLMs are extremely sensitive to subtle changes in prompt formatting in few-shot settings, with performance differences of up to 76 accuracy points when evaluated using LLaMA-2-13B. Sensitivity remains even when increasing model size, the number of few-shot examples, or performing instruction tuning. Our analysis suggests that work evaluating LLMs with prompting-based methods would benefit from reporting a range of performance across plausible prompt formats, instead of the currently-standard practice of reporting performance on a single format. We also show that format performance only weakly correlates between models, which puts into question the methodological validity of comparing models with an arbitrarily chosen, fixed prompt format. To facilitate systematic analysis we propose FORMATSPREAD, an algorithm that rapidly evaluates a sampled set of plausible prompt formats for a given task, and reports the interval of expected performance without accessing model weights[1]. Furthermore, we present a suite of analyses that characterize the nature of this sensitivity, including exploring the influence of particular atomic perturbations and the internal representation of particular formats.

1 INTRODUCTION

As the capabilities of LLMs have rapidly improved, their sensitivity to input prompt features has been used to optimize performance via prompt engineering (White et al., 2023). However, there has been little work in characterizing this sensitivity, especially to seemingly innocuous feature choices that preserve prompt meaning and intent. In this work, we analyze the sensitivity of widely used, open-source LLMs to a class of features that should not influence a prompt's interpretation: formatting choices. We find that pre-trained LLMs are sensitive to these choices in unpredictable ways, with accuracy varying in up to 76 points for LLaMA-2-13B between equivalent formats, and $\sim$10 accuracy points on average across 50+ tasks and several models. We also show that this variance is not eliminated by adding few-shot examples, increasing model size, or instruction tuning.

Designing prompt templates is a critical part of effectively using a pre-trained language model. This design process includes making choices about wording, choosing few-shot examples for in-context learning, and making decisions about seemingly trivial features like formatting. This process, and often even the resulting templates, is rarely reported or discussed in research papers, under the assumption that performance variance across these choices is insignificant compared to variance across data points or models. However, some anecdotal evidence points to formatting choices actually having a significant influence on model behavior (Aghajanyan, 2023). In some cases, researchers report a limited number of manually generated formats to show that scaling trends hold despite perfor-

[1]FORMATSPREAD's code can be found at https://github.com/msclar/formatspread.

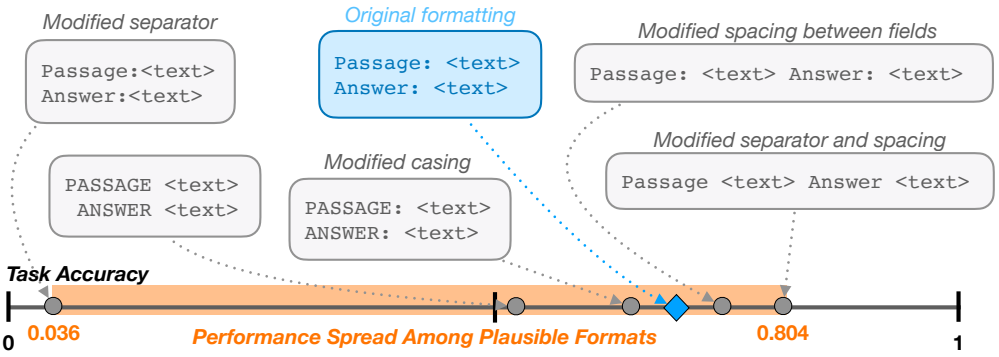

Figure 1: Slight modifications in prompt format templating may lead to significantly different model performance for a given task. Each `<text>` represents a different variable-length placeholder to be replaced with actual data samples. Example shown corresponds to 1-shot LLaMA-2-7B performances for task280 from SuperNaturalInstructions (Wang et al., 2022). This StereoSet-inspired task (Nadeem et al., 2021) requires the model to, given a short passage, classify it into one of four types of stereotype or anti-stereotype (gender, profession, race, and religion).

mance being significantly different (Schick et al., 2021). The assumption that formatting does not influence overall model performance may become problematic when improvements over existing approaches are attributed to the amount and source of training data, number of parameters, or model architecture, without also accounting for changes in prompt format. Ignoring variance across formats may also negatively affect user experience, e.g. if users inadvertently choose formats the LLM does not perform well on.

Our proposed tool, FORMATSPREAD, enables a systematic analysis of these variances across a wide set of semantically equivalent prompt formats within a user-specified computational budget. We find that choices in formatting few-shot examples during in-context learning introduce spurious biases that may lead to significantly different conclusions in model performance. The sensitivity to formatting choices that we discover across widely-used, open-source models suggests that future research would benefit from reporting a performance *spread* over a sufficient sample of plausible formats, instead of simply reporting the formatting used and its performance, as is currently standard. Moreover, we argue that this reporting is crucial when comparing the performance of different models, as we show the influence of formatting choices only weakly correlates between models, thus making and fixing a formatting choice could introduce a significant confounding factor.

Fully exploring the space of prompt formats is intractable, as computation costs scale linearly with the number of formats considered. FORMATSPREAD efficiently explores the space of prompt formats under a user-specified computational budget using Bayesian optimization. FORMATSPREAD does not require access to the model weights, allowing its use on API-gated models: we find a spread up to 56 accuracy points with a median spread of 6.4 accuracy points with GPT3.5 across 320 formats and 53 tasks at a cost of under 10USD on average per task. Beyond facilitating evaluation, we also propose a suite of analyses to further characterize model sensitivity to formatting. Among other results, we show that the separability of continuous prompt embeddings correlates with the spread observed in task performance.

2 OVERVIEW

We evaluate LLM performance over the space of prompt formats that may plausibly be chosen by a non-adversarial user when designing a prompt for a target task, where the space of formats is defined by a grammar (§3.1). Our grammar's definition naturally induces a definition of semantic equivalence among formats. We quantify model sensitivity in terms of performance range in a target task across the space of equivalent prompt formats to the original choice (§4.2). We cast the problem of searching across this space as a bandit problem, and propose FORMATSPREAD (§3), which consists of a grammar (§3.1) and a procedure to estimate the minimum and maximum performance across a set of semantically equivalent formats given a pre-defined metric (§3.2). FORMATSPREAD uses Bayesian optimization to identify the expected performance range with low additional computational cost (§4.5) all without requiring access to model weights, which enables use on API-gated

LLMs. Furthermore, we perform in-depth analysis of this observed sensitivity, including by quantifying the contribution of individual feature choices to the final performance (§4.3) and measuring the identifiability of a format based solely on a model's internal, continuous representation of any prompt via correlation with model performance (§4.4).

3 MEASURING SENSITIVITY WITH FORMATSPREAD

3.1 GRAMMAR OF PLAUSIBLE PROMPT FORMATS

We construct a grammar that defines both the space of plausible prompt formats and semantic equivalence between formats. The grammar is manually constructed, as opposed to automatically induced from data, to guarantee a higher level of precision when defining the set of equivalent formats. Our grammar is directly tested by verifying that it can generate the formatting associated with 100+ Super-NaturalInstructions tasks (Wang et al., 2022).

Our grammar consists of fields that are composed to create a prompt format. For example, the format `'Passage: <text> || Answer: <text>'`, has basic fields `'Passage: <text>'`, and `'Answer: <text>'`, denoted a_1, and a_2. Each basic field consists of a *descriptor* (e.g. `'Passage'`), a *separator* (e.g. `': '`), and a text placeholder to replace with each data point. We define basic fields as $B_1(d, s, f) := f(d)s$`<text>` using Backus-Naur notation, where d is a descriptor string, $s \in \mathcal{S}_1$ a separator, and $f \in \mathcal{F}_{\text{casing}}$ a function that alters d while preserving meaning. Thus, in our example, $a_1 = B_1($`Passage`$,$`': '`$, id)$ and $a_2 = B_1($`Answer`$,$`': '`$, id)$, with id the identity function. We define joining several fields as $B_2^{(n)}(X_1, \ldots, X_n, c) := X_1 c X_2 c \ldots c X_n$, with $c \in \mathcal{C}$ being a *space*. Our example's prompt format may be written as $B_2^{(2)}(a_1, a_2,$ `' || '`$)$.

The grammar also supports enumeration, which is defined as joining several basic fields, each representing a different list item. For example, the enumeration `'Option (A): <text>, Option (B): <text>, Option (C): <text>'` may be written as $B_2^{(3)}(a_1, a_2, a_3,$ `' || '`$)$, where $a_i = B_1(e_i,$ `': '`$, id)$. In our example, e_1 represents `'Option (A)'`, and may in turn be written as the concatenation $e_i := d s_2 f_{\text{item}}(i)$ with $d =$ `'Option'`, $s_2 =$ `' '` (single space), and $f_{\text{item}}(1) =$ `'(A)'`. Each f_{item} transforms an item i using a number format (e.g. letters or Roman numerals, denoted as $\mathcal{F}_{\text{item2}}$) and an item wrapper (e.g. `(A)` or `[A]`, denoted as $\mathcal{F}_{\text{item1}}$).

In summary, we define valid prompt formats as those accepted by the following grammar:

$$
\begin{aligned}
B_0() &:= \texttt{<text>} \\
B_0'(d, s) &:= f(d)s \quad \text{with } s \in \mathcal{S}_1,\ f \in \mathcal{F}_{\text{casing}} \\
B_1(d, s, f) &:= f(d)s\texttt{<text>} \quad \text{with } s \in \mathcal{S}_1,\ f \in \mathcal{F}_{\text{casing}} \\
B_2^{(n)}(X_1, \ldots, X_n, c) &:= X_1 c \ldots c X_n \quad \text{with } c \in \mathcal{C},\ X_i \in \{B_0, B_0', B_1, B_2, B_3\}\ \forall i \\
B_3^{(n)}(d, j_1, \ldots, j_n, s_1, s_2, c, f_1, f_2) &:= B_2^{(n)}(B_1(e_1, s_1, f_2)), \ldots, B_1(e_n, s_1, f_2), c) \\
&\qquad \text{where } e_i := f_2(d)\ s_2\ f_1(j_i),\ j_i \in \mathbb{N}_0\ \forall i, \\
&\qquad s_1 \in \mathcal{S}_1, s_2 \in \mathcal{S}_2, f_1 \in \mathcal{F}_{\text{item}}, f_2 \in \mathcal{F}_{\text{casing}}
\end{aligned}
$$

Our grammar defines valid formats as finite compositions of B_0, B_0', B_1, B_2, B_3. The sets $\mathcal{S}_1, \mathcal{S}_2, \mathcal{C}$, $\mathcal{F}_{\text{casing}}$, $\mathcal{F}_{\text{item}}$ (two sets of separators, spaces, casing functions, and itemizing functions respectively) are pre-defined by the user. Throughout this work, we instantiate all sets with values typically observed in human-written prompt formats. We intentionally only modify the casing of descriptors (via $\mathcal{F}_{\text{casing}}$) to guarantee semantic equivalence; one may also define a set of functions that paraphrases the descriptor, e.g., via synonym replacement. Appendix A.2 contains the full list of values we use for the constant sets, as well as a visualization of a prompt template generated from the grammar.

Prompt Format Equivalence. Two prompt formats p_1, p_2 are equivalent if they represent the same rule application B_i, the descriptors (if any) are the same, and the sub-elements (if any) are equivalent. Appendix A.1 contains the formal definition of equivalence. The grammar's strict definition allows us to assume that sets of equivalent formats share equivalent meanings. When measuring sensitivity (§3.2), we explore only the space of formats equivalent to a task's original format.

Contextual Restrictions. We define restrictions to the combinations of spaces and separators to further ensure naturalness. For example, if $B_2(X_1, \ldots, X_n, c)$ where c does not contain a newline, then each X_i's separators and any subcomponents' separators should not contain a newline. This